HYTech LMS — Comprehensive Audit Report

System: HYTech Learning Management System · **Organization:** HYT Global Institute
Audit date: 30 July 2026 · **Auditor:** Automated technical audit
Commit under audit: `e0a4d5f` — *Pin firebase-admin to 13.x and verify Functions load in QA*
Branch: `security/hardening-remediation` · **Committed:** 2026-07-29 15:12 +0800
Classification: Internal technical documentation

> **Scope note.** Secret exposure was assessed against **git-tracked content at this commit only**, as requested. Git *history* was not audited — historical credential exposure remains an open item in `HYTech/docs/TURNOVER_RISK_REGISTER.md` and is unaffected by this report. Eight files were dirty in the working tree at audit time; findings below cite committed content unless stated otherwise.

---

1. Executive summary

Overall: the application is in good shape. The delivery pipeline is not.

Every functional test suite passes — **92 of 92 tests green** across unit, rules, and three end-to-end suites, on a clean production build with Cloud Functions loading successfully. No credential material of any kind is committed: no private keys, no service-account JSON, no OAuth or CI tokens, and no `.env` files. Secret handling through GitHub Secrets is correct and consistent.

One issue is **blocking production releases right now**:

> 🔴 `npm run security:secrets` **fails at this commit.** It is a hard, non-`continue-on-error` step in `qa.yml`, and `deploy.yml` requires QA to conclude `success` before it will deploy. Production therefore cannot ship through the normal path. The failure is a **false positive** — the repository's own secret scanner is matching the historical credential that `TURNOVER_RISK_REGISTER.md` documents in prose.

Severity	Count	Items
🔴 Critical	1	Secret-scan false positive blocks the release pipeline
🟠 High	1	Firebase web keys and project identifiers inlined in workflow YAML
🟡 Medium	2	Shallow test coverage of highest-risk logic; unblocking dependency advisory
🔵 Low	5	No lint gate, stale committed build output, no URL protocol allowlist, log noise, god files
✅ Verified clean	11	See §3.2 (4 items) and §8 (7 items)

Change since the 23 July audit: the CSP moved from report-only to enforcing, and QA now gates deploy — both genuine improvements. The pipeline break is new, introduced by this same commit.

---

2. Scope and method

Executed

Check	Command	Environment
Repository secret scan	`npm run security:secrets`	Local, repo scanner
Independent credential scan	13 pattern families over `git ls-files`	164 tracked files
Ignore-rule verification	`git check-ignore`	3 local env files
Production build	`npm run build`	Dummy env, as CI
Staging build	`npm run build:staging`	`.env.staging.local`
Functions syntax + runtime load	`node --check` + `require()`	Node 22
Unit tests	`npm run test:unit`	Vitest 4
Authorization rules tests	`npm run test:rules`	Firestore emulator, JDK 21
E2E public + accessibility	`npm run test:e2e:public`	Chromium, local preview
E2E responsive	`npm run test:e2e:responsive`	Chromium, 9 viewports
E2E authenticated roles	`npm run test:e2e:roles`	Chromium, staging backend
Dependency audit	`npm audit --omit=dev`	App + Functions
Code health metrics	Static analysis	74 files, 37,565 LOC

Not executed — and why

- **Git history scan.** Explicitly out of scope per the request.
- **Google Cloud console review.** No console access from this environment, so API-key restrictions, App Check enforcement state, Firestore backup/PITR settings, and IAM bindings are **unverified**. Several findings below depend on these; they are marked *requires console verification*.
- **Firefox, WebKit, and Edge E2E.** CI covers all four browsers; this audit ran Chromium only.
- **Penetration testing** of live endpoints, and **restore drill** — both remain open items in the risk register.

3. Secret exposure audit

3.1 Findings

Thirteen credential pattern families were applied across 164 tracked files.

No credential material is committed. Every family targeting a genuinely secret value returned **zero** matches: PEM private keys, service-account JSON (`"type": "service_account"` and `"private_key"`), Google OAuth refresh tokens, GitHub tokens, Slack tokens, AWS access key IDs, npm tokens, JWTs, and inlined `FIREBASE_TOKEN` values.

Two families produced matches, four in total. One — a generic `password` / `secret` / `api_key` assignment heuristic — fired twice inside the stale committed build bundle, and both were confirmed to be React internals and a form state object, i.e. false positives (see §3.2). The other family accounts for the remaining two matches, which share the **same class of value**:

File	Line	Value	Project
`.github/workflows/deploy.yml`	59	`AIzaSy…m6-M`	`hyt-global-institute-lms` (production)
`.github/workflows/qa.yml`	90	`AIzaSy…0hBg`	`hytech-lms-staging` (staging)

Alongside each key, the full web configuration is inlined: auth domain, project ID, storage bucket, messaging sender ID, and app ID for both environments.

These are Firebase Web API keys, which are public by design. They are compiled into the browser bundle and are visible to anyone who loads the site. They are project *identifiers*, not authorization secrets — security rests on Authentication, Security Rules, App Check, and backend validation. Both workflow files carry a comment saying exactly this, and that assessment is correct. **This is not a credential leak.** See §4 for the real concern.

3.2 Verified clean

- **No `.env` file is tracked.** Only `.env.e2e.example` and `.env.staging.example` are committed, and both contain placeholders.
- **Local secret files are correctly ignored.** `git check-ignore` confirms `.env.local`, `.env.staging.local`, and `.env.e2e.local` are all ignored; `.gitignore` covers `.env`, `.env.local`, `.env.*.local`, and `*.local`.
- **Every genuine secret is referenced, never inlined.** `FIREBASE_TOKEN`, `FIREBASE_SERVICE_ACCOUNT_JSON`, and all twelve `E2E_*` credential values resolve from `${{ secrets.* }}`.
- **The committed build artifact contains no credentials.** `HYTech/build/` holds a stale 411 KB bundle; it was searched for API keys, `apiKey` / `projectId` fields, and Firebase hostnames — all absent. (Its presence is a separate hygiene finding: §6.2.)

---

4. Findings register

🔴 CRITICAL-1 — Secret-scan false positive blocks the release pipeline

Evidence. `npm run security:secrets` exits 1 at this commit:

```
Potential committed secrets found:
HYTech\docs\TURNOVER_RISK_REGISTER.md: legacy published password
```

The matched line is documentation, not a credential:

```
TURNOVER_RISK_REGISTER.md:65
credential pairs across two domains (`admin@hytech.com/admin1234` and
```

`scripts/check-no-secrets.mjs` pattern `legacy published password` is `/\b(?:admin|trainer|supervisor|student)123[4]?\b/i`. The risk register describes the historical leak that this very pattern exists to detect, so the scanner flags the document that records the incident.

Impact. In `qa.yml` the step *"Scan for blocked secret patterns"* has no `continue-on-error` and runs **before** unit tests, so the entire QA workflow fails immediately on every push and pull request. Because `deploy.yml` triggers on `workflow_run` and requires `github.event.workflow_run.conclusion == 'success'`, **production cannot deploy through the normal path.** Only `workflow_dispatch` — the rollback escape hatch, which bypasses QA — can currently ship. The commit that introduced the register (`e0a4d5f`) is the commit that broke the gate.

Recommendation. Narrow the detector rather than weakening it. Either exclude documentation paths that are allowed to describe past incidents, or require the pattern to appear in credential-like syntax rather than prose:

```
// scripts/check-no-secrets.mjs
const allowlist = [/^HYTech[\\/]docs[\\/]TURNOVER_RISK_REGISTER\.md$/];
// ...then skip a file when a documented-incident allowlist entry matches.
```

Add a regression test asserting the scanner exits 0 on a clean tree, so the gate cannot silently break the pipeline again. **Do not** simply delete the pattern — it is guarding a real historical exposure.

🟠 HIGH-1 — Web keys and project identifiers inlined in workflow YAML

Evidence. §3.1. Production and staging API keys plus complete web configs are hardcoded in `deploy.yml` and `qa.yml`.

Why this is a finding even though the values are public. Three reasons:

1. **Restriction state is unverified.** An unrestricted Firebase Web API key permits Identity Toolkit calls from any origin — account-creation spam, quota consumption, and password-reset probing. `config/appSettings` defaults `allowSelfRegistration` to **true**, so an unrestricted key plus open self-registration is a usable abuse path. *Requires console verification:* confirm HTTP-referrer restrictions and per-API restrictions on both keys.
2. **The repository is internally inconsistent.** `VITE_RECAPTCHA_SITE_KEY` is equally public yet is passed via `${{ vars.VITE_RECAPTCHA_SITE_KEY }}` in both workflows. Two conventions for the same class of value obscures which values are centrally managed.
3. **Rotation requires a code change.** Rotating a key today means editing YAML and opening a pull request, rather than updating a repository variable.

Recommendation. Move all six `VITE_FIREBASE_*` values per environment into GitHub Actions **variables** (not secrets — they are not secret), matching the existing reCAPTCHA pattern. Then verify and document key restrictions in the console. Neither change alters runtime behaviour.

🟡 MEDIUM-1 — Test coverage does not reach the highest-risk logic

Evidence. All suites pass, but what they cover is narrow:

Suite	Tests	Actual coverage
`tests/unit/`	8	`csv.test.js` **only** — CSV helpers
`tests/rules/`	10	`firestore.rules.test.js` only — **no Storage rules tests**
`tests/e2e/roles.spec.js`	17	14 are *"loads without page overflow"*; 3 are role isolation
`tests/e2e/public.spec.js` + a11y	30	Route guards, redirects, axe checks
`tests/e2e/responsive.spec.js`	27	Viewport fit across 9 sizes

Nothing automated exercises the subsystem the turnover documentation itself identifies as most dangerous:

- `submitAssessmentAttempt` grading, scoring, and attempt immutability
- the availability-window / deadline resolution rule, including the legacy bare `YYYY-MM-DD` end-of-day vs start-of-day distinction applied in **two** places
- dual-collection resolution across `assessments` and `assignments`
- `graduateEnrollment` requirement checks and the certificate transaction
- `storage.rules` — 3.7 KB of policy with zero tests, despite the emulator for it being started by `test:rules`

Impact. A regression in grading or deadline handling ships green. These paths produce academic records that are immutable by design, so a defect is expensive to unwind.

Recommendation. Priority order: (1) unit-test the deadline/date resolution helper against both ISO and legacy formats; (2) add rules tests for `storage.rules`; (3) add emulator-backed Functions tests for `submitAssessmentAttempt` covering closed windows, attempt limits, `oneResponsePerUser`, and paragraph → `pending_review`.

🟡 MEDIUM-2 — Dependency advisory that can never block, on a control that does not apply

Evidence.

```
[HIGH] react-router  installed 7.18.2 (range 7.12.0 - 8.2.0)
  React Router: RSC Mode CSRF Bypass Allows Action Execution Before 400 Response
  GHSA-qwww-vcr4-c8h2 - vulnerable: >=7.12.0 <8.3.0
  fixAvailable: react-router-dom 7.11.0 (isSemVerMajor: true)
[HIGH] react-router-dom  >=7.12.0-pre.0
```

Functions dependencies: **0 vulnerabilities.**

Assessment — not exploitable in this configuration. The advisory requires React Router **RSC mode**. This application uses the classic declarative API (`BrowserRouter` / `Routes` / `Route` in `src/App.jsx:3`), has no `createBrowserRouter`, no `loader:` or `action:` handlers, no SSR, and no RSC plugin in `vite.config.js`. There is no server-side action to bypass. The offered "fix" is a **semver-major downgrade to 7.11.0**, which would be a regression, not a remediation.

The real finding is the gate, not the CVE. `qa.yml` marks the audit step `continue-on-error: true`, so no dependency advisory — including a future one that *is* exploitable — can ever fail CI.

Recommendation. Keep 7.18.2. Record a dated disposition in `DEPENDENCY_RISK_REGISTER.md` citing the RSC-mode precondition, and upgrade to ≥ 8.3.0 on the normal cadence when the React 19 / Router 8 migration is planned work. Separately, decide deliberately whether the audit step should block; if it should stay advisory, say so in a comment so the next reader knows it is intentional.

🔵 LOW findings

LOW-1 — No lint or static-analysis configuration exists. No ESLint, Biome, or Prettier config is present anywhere in `HYTech/`. There is consequently no lint gate in CI, which is already engineering priority #9 in the turnover guide. For a 37,565-line codebase with no type checking, this is the cheapest available quality win.

LOW-2 — Stale build output is committed. `HYTech/build/` has **9 tracked files** including a 411 KB `main.cd94a9b5.js` — Create React App output naming, from before the migration to Vite (`dist/`). `.gitignore` lists `build/`, but ignore rules do not untrack already-committed files. Verified to contain no credentials, but it is dead weight that can be mistaken for current output, and any future commit there could bake in environment values. Fix: `git rm -r --cached HYTech/build`.

LOW-3 — No protocol allowlist on user-supplied URLs. Material links render straight into `href`, e.g. `href={selectedMaterial.link}` (`StudentCourse.jsx:2979`, `ClassDetail.jsx:3760`). No `http(s)`-only validation exists anywhere in `src/utils/`. A trainer, co-trainer, or admin could store a `javascript:` URL that executes in a trainee's session on click; React warns on such URLs but does not block them.

Currently mitigated by the enforcing CSP — `script-src 'self' https://www.google.com https://www.gstatic.com` has no `'unsafe-inline'`, which blocks `javascript:` URI execution on Hosting. **But** the mitigation is absent in local development (the Vite dev server sets no CSP), and it makes CSP strictness load-bearing for a defect that belongs in input validation. Add an explicit scheme allowlist at write time and at render time.

LOW-4 — Log noise. 325 `console.log` / `warn` / `error` / `debug` calls in `src/`, and 29 `TODO` / `FIXME` / `HACK` markers across `src/` and `functions/src/`. Console output in production risks incidental disclosure of identifiers and object shapes; strip or gate it behind a debug flag.

LOW-5 — Maintainability concentration. `ClassDetail.jsx` 7,712 lines; `firestoreService.js` 5,149; `StudentCourse.jsx` 3,981; `functions/src/index.js` 1,949. Four files carry most of the product behaviour. Already tracked as engineering priority #6; restated here because it is the root cause of the coverage gap in MEDIUM-1 — these files are too large to test meaningfully.

---

5. QA results

Full run at commit `e0a4d5f`. **92 functional tests, 92 passed, 0 failed.**

Suite	Result	Detail
Production build	✅ PASS	Built in 1.94 s; code-split, largest chunk `firebase-firestore` 404.69 kB (115.53 kB gzip)
Staging build	✅ PASS	Built in 1.39 s
Functions syntax	✅ PASS	`node --check` clean
Functions runtime load	✅ PASS	`require()` succeeds — confirms the `firebase-admin` 13.x pin
Unit tests	✅ PASS	8/8, 523 ms
Firestore rules tests	✅ PASS	10/10, 4.89 s, emulator (JDK 21)
E2E public + accessibility	✅ PASS	30/30, 15.4 s — includes axe checks on `/` and `/signup` with no serious or critical violations
E2E responsive	✅ PASS	27/27, 11.8 s — 9 viewports, 390×844 to 1920×1080
E2E authenticated roles	✅ PASS	17/17, 18.7 s — admin, trainer, student; role isolation enforced
Secret scan	❌ **FAIL**	**CRITICAL-1 — blocks the pipeline**
Dependency audit (app)	⚠️ 2 high	MEDIUM-2 — not applicable to this configuration
Dependency audit (functions)	✅ PASS	0 vulnerabilities

Notable positives observed during the run: unauthenticated access to all 24 role-scoped routes correctly redirects to sign-in; unknown routes return safely to the landing page; and each role is prevented from remaining on another role's dashboard.

6. Code health metrics

Metric	Value
Source files (`src/` , `.js` / `.jsx`)	74
Source lines	37,565
Largest component	`ClassDetail.jsx` — 7,712 lines
Data-access layer	`firestoreService.js` — 5,149 lines
Cloud Functions	`functions/src/index.js` — 1,949 lines (5 triggers, 16 callables)
Security rules	`firestore.rules` — 579 lines; `storage.rules` — 3.7 KB
`console.*` calls	325
`TODO` / `FIXME` / `HACK`	29
`dangerouslySetInnerHTML`	**0** ✅
`target="_blank"` anchors	7 — **all** carry `rel="noopener noreferrer"` ✅
Lint configuration	**none**
Tracked files	164

7. Hosting security headers

Verified in `HYTech/firebase.json`. The Content-Security-Policy is **enforcing** (header key `Content-Security-Policy`, not `-Report-Only`) — a change since the 23 July audit, and the mitigation LOW-3 currently relies on.

```
default-src     'self'
script-src      'self' https://www.google.com https://www.gstatic.com
style-src       'self' 'unsafe-inline'
img-src         'self' data: blob: https://firebasestorage.googleapis.com
                https://storage.googleapis.com https://lh3.googleusercontent.com
font-src        'self' data:
connect-src     'self' https://*.googleapis.com https://*.firebaseio.com
                wss://*.firebaseio.com https://*.cloudfunctions.net https://www.google.com
frame-src       https://www.google.com
frame-ancestors 'none'
base-uri        'self'
form-action     'self'
object-src      'none'
```

Also set: HSTS (`max-age=31536000; includeSubDomains; preload`), `X-Frame-Options: DENY`, `X-Content-Type-Options`, Referrer-Policy, Permissions-Policy (camera, microphone, geolocation, payment, USB, and interest-cohort all disabled), and COOP.

`style-src 'unsafe-inline'` is required by the Tailwind/React inline-style approach and is the one deliberate relaxation. `script-src` correctly omits `'unsafe-inline'`, which is what makes the policy meaningful.

---

8. Authorization posture — verified by inspection

These controls were confirmed present in `firestore.rules` at this commit:

- **Server-write-only, no client path exists:** `assessments/*/attempts`, `assignments/*/attempts`, `students/{uid}/progress/{classId}`, `certificates`, `securityLogs`, `classDirectory`, and course-template answer keys. Attempts and certificates are the anti-tampering boundary of the grading model.
- **Answer keys** live in private child documents, unreadable by class members.
- **PII segregation:** phone, address, birth date, and emergency contact live in `users/{uid}/private/profile`, outside the directory-readable document.
- **Privilege escalation blocked:** a self-registering user cannot set `role`, `status`, or `createdBy`; self-update is restricted to an explicit key allowlist. The `createdBy == 'admin'` email-verification waiver in `canUseLms()` is correctly unreachable from self-registration.
- **Notification abuse blocked:** students may notify staff only.
- **Deletes disabled in rules** for sectors, courses, classes, and assessments — routed through cascading `delete*Secure` callables so answer keys and attempts cannot be orphaned.
- **Client activity logging** is restricted to five allowlisted actions.

---

9. Remediation plan

#	Action	Severity	Effort	Blocks release?
1	Fix the secret-scanner false positive; add a scanner regression test	🔴 Critical	~1 h	**Yes — do first**
2	Verify API-key restrictions in Google Cloud for both projects	🟠 High	~1 h	No, but do this week
3	Move `VITE_FIREBASE_*` into GitHub Actions variables	🟠 High	~1 h	No
4	Record the react-router RSC disposition in the dependency register	🟡 Medium	~30 m	No
5	Decide whether `npm audit` should block, and comment the intent	🟡 Medium	~30 m	No
6	Unit-test deadline/date resolution (ISO + legacy formats)	🟡 Medium	~4 h	No
7	Add `storage.rules` tests	🟡 Medium	~4 h	No
8	Add emulator tests for `submitAssessmentAttempt`	🟡 Medium	~1–2 d	No
9	Add a URL scheme allowlist at write and render time	🔵 Low	~2 h	No
10	`git rm -r --cached HYTech/build`	🔵 Low	~5 m	No
11	Add ESLint with a CI gate	🔵 Low	~4 h	No
12	Strip or gate the 325 `console.*` calls	🔵 Low	~2 h	No
13	Continue splitting the four oversized modules	🔵 Low	ongoing	No

Items 1–3 are the release-critical set. Items 6–8 are the ones that would have caught a real defect.

10. Sign-off

Release recommendation: do not deploy until CRITICAL-1 is fixed — not because the application is unsafe, but because the only currently available deploy path is the QA-bypassing `workflow_dispatch` escape hatch. Shipping that way discards the gate the previous hardening work just installed.

With CRITICAL-1 resolved, this commit is in a releasable state on the evidence available locally: clean builds, Functions loading, 92 passing tests, no committed credentials, strong enforced headers, and an authorization model whose critical write paths have no client access.

The findings this audit cannot close are the ones needing console access — API key restrictions, App Check enforcement state, backup/PITR configuration, and IAM. Those remain owned by `HYTech/docs/TURNOVER_RISK_REGISTER.md` and must be verified against the live projects before production responsibility transfers.

Audit trail

Item	Value
Commit	`e0a4d5ff5913eb4537c3520620d631236fa49dbe`
Branch	`security/hardening-remediation`
Tracked files scanned	164
Credential pattern families applied	13
Functional tests executed	92 (92 passed)
CI gates evaluated	2 (1 failing)
Supersedes	`HYTech_LMS_Comprehensive_Audit_Report_2026-07-23`

Prepared 30 July 2026 against commit `e0a4d5f`. Git history and live Firebase console state were out of scope. Re-verify the risk register against the live projects before any operational change.